



الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يُونِيسْكَوِيَّتِي اِسْلَامُ اِنْتَارَايْغُسِيَا مِلْدِسِيَا
Garden of Knowledge and Virtue

PROJECT REPORT

TRACK 1 - KAGGLE COMPETITION

THE MARS ENVIRONMENTAL DYNAMICS ANALYZER (MEDA) VIRTUAL SENSOR
RECOVERY COMPETITION

MCTA 4363

DEEP LEARNING

LECTURER: DR. HASAN FIRDAUS BIN MOHD ZAKI

SEMESTER 2 2025/2026

TEAM MEMBERS	MATRIC NUMBER
MUHAMMAD AMMAR ZUHAIR BIN NOR AZMAN SHAH	2110259
MUHAMMAD BASIL BIN ABDUL HAKIM	2215315
MUHAMMAD SYAFIQ BIN NOR AZMAN	2213187

TABLE OF CONTENTS

1. INTRODUCTION.....	2
1.1 Project Overview.....	2
1.2 Problem Statement.....	2
1.3 Objectives.....	2
2. BACKGROUND.....	3
2.1 Problem Domain.....	3
2.2 Review of Deep Learning for Time-Series Forecasting.....	3
2.3 Existing Work.....	4
3. DATASET.....	5
3.1 Data Source and Description.....	5
3.2 Data Preprocessing.....	5
4. METHODOLOGY.....	6
4.1 Overall Pipeline.....	6
4.2 Baseline Model.....	6
4.3 Primary Model Architectures.....	7
4.4 Training Configuration.....	9
4.5 Evaluation Metrics.....	9
5. EXPERIMENTAL SETUP.....	10
5.1 Computing Environment.....	10
5.2 Experiment Log.....	10
5.3 Hyperparameter Tuning.....	10
6. RESULTS AND ANALYSIS.....	12
6.1 Model Performance Comparison.....	12
6.2 Kaggle Leaderboard Standings.....	12
6.3 Prediction Visualisations.....	13
6.4 Error Analysis.....	13
6.5 Discussion.....	14
7. REFLECTION.....	15
7.1 Challenges Encountered.....	15
7.2 Lessons Learned.....	15
8. CONCLUSION AND FUTURE WORK.....	16
8.1 Conclusion.....	16
8.2 Future Work.....	16
REFERENCES.....	17

1. INTRODUCTION

1.1 Project Overview

This project focuses on predicting atmospheric pressure on Mars using a multivariate time-series dataset provided through a Kaggle competition. The dataset consists of environmental, rover, orientation, and sensor-related variables recorded at very high frequency. The target variable for prediction is pressure.

The project was conducted as a machine learning forecasting task, where different models were developed and compared to determine the most suitable approach for predicting Martian atmospheric pressure.

The main motivation behind this project is the importance of accurate Martian atmospheric prediction. Weather forecasting on Mars is relevant for robotic missions, scientific planning, and future human exploration. Since atmospheric pressure on Mars follows temporal patterns influenced by environmental and solar cycles, this task is suitable for time-series forecasting methods.

1.2 Problem Statement

The main problem addressed in this project is the prediction of Martian atmospheric pressure from a large sequential dataset. The dataset contains approximately 9 million observations, with measurements recorded at millisecond intervals. This creates several challenges.

First, the dataset is extremely large, which increases computational cost and memory usage during model training. Second, because the data is recorded at millisecond frequency, adjacent rows are highly correlated. Therefore, although the dataset contains millions of rows, not all rows provide independent information. Third, several sensor columns contain missing values, requiring proper preprocessing before training. Finally, the prediction task involves temporal dependencies, meaning that the model must learn patterns over time rather than treating each row independently.

1.3 Objectives

1. To perform data cleaning, feature engineering, and data transformation techniques suitable for machine learning applications.
2. To explore and implement different machine learning and deep learning approaches for atmospheric pressure prediction.
3. To establish an experimental framework for training and evaluating predictive models on Martian environmental data.

2. BACKGROUND

2.1 Problem Domain

Mars atmospheric prediction is an important research area due to the planet's complex environmental conditions. Atmospheric pressure on Mars is affected by solar heating, dust activity, seasonal variation, and local atmospheric dynamics. Unlike simple tabular prediction tasks, atmospheric prediction involves temporal behaviour, where past observations may influence future or current pressure values.

In this project, the dataset contains variables such as local Mars time, solar angle, rover position, rover velocity, rover pitch, yaw, roll, irradiance values, sensor flags, and pressure. These variables represent both environmental and rover-related conditions that may influence atmospheric pressure readings.

A major characteristic of this dataset is its high sampling frequency. Since observations are recorded every millisecond, neighbouring observations are very similar. This makes the dataset large but highly redundant. As a result, model design must consider not only the number of rows, but also the actual time span represented by each input sequence.

2.2 Review of Deep Learning for Time-Series Forecasting

Time-series forecasting involves predicting a target value based on ordered observations over time. In this project, several models were considered, including MLP, XGBoost, LSTM, and TCN.

A Multi-Layer Perceptron (MLP) is a feedforward neural network consisting of input, hidden, and output layers. MLP models are commonly used as baseline neural network models because they can learn nonlinear relationships between input features and the target variable.

eXtremely Gradient Boosting (XGBoost) is a gradient boosting algorithm that performs well on structured tabular data. It builds an ensemble of decision trees and can model nonlinear relationships between input features and the target variable. Due to its strong predictive capability and flexibility, XGBoost has been widely applied in regression, classification, and forecasting tasks.

Long Short-Term Memory (LSTM) is a recurrent neural network architecture designed to handle sequential data. It uses memory cells and gating mechanisms to retain and update information from previous timesteps. LSTM is commonly used for time-series forecasting because it can learn temporal dependencies and capture patterns that evolve over time.

Temporal Convolutional Network (TCN) is a convolution-based architecture for time-series forecasting. It uses one-dimensional causal convolutions to process temporal sequences while preserving the temporal order of observations. TCN also employs dilated convolutions to expand its receptive field, enabling the model to capture both short-term and long-term temporal patterns efficiently.

2.3 Existing Work

Kaggle competitions provide a practical environment for comparing machine learning models on hidden test data. In this competition, the objective was to predict Martian atmospheric pressure from a large sensor-based dataset. The leaderboard score was used to compare model performance among teams.

Uprety et al. (2025) has shown that machine learning can be applied to predict atmospheric variables such as pressure, temperature, wind, and dust-related measurements. They compared multiple time-series models, including RNN-based models, TCN, Transformer-based models, N-BEATS, and TiDE. The findings suggested that TCN and TiDE were particularly effective for pressure and temperature forecasting because they were able to capture realistic temporal patterns.

This project builds on the idea that Martian pressure prediction is a temporal forecasting problem. Instead of relying only on tabular models, sequence-based deep learning models were explored to capture the time-dependent structure of the data.

3. DATASET

3.1 Data Source and Description

The dataset was provided through a Kaggle competition. The training and testing files were stored in Parquet format, which is suitable for large datasets because it is compressed and efficient for column-based storage.

The training dataset contains approximately 9 million rows. Each row represents a recorded observation at millisecond-level frequency. The target variable is PRESSURE, while the input variables include time-related features, rover movement and orientation features, solar angle features, irradiance features, humidity-related features, and sensor status indicators. However, some columns contained a high percentage of missing values.

3.2 Data Preprocessing

The preprocessing stage was necessary to convert the raw dataset into a suitable format for model training.

First, missing values were analyzed by calculating the percentage of missing entries for each column. Columns with very high missing percentages were removed. For the remaining features, missing values were filled using linear interpolation, followed by forward filling and backward filling. Linear interpolation estimates missing values based on neighbouring observations, which is appropriate for sequential data where sensor readings are expected to change gradually.

Second, cyclical time features were extracted from the Local Mean Sidereal Time (LMST) column. Since time is periodic, direct numerical representation may not capture the circular nature of daily cycles. Therefore, sine and cosine transformations were applied to encode time. Additional second-harmonic time features were also considered to represent semi-daily pressure patterns.

Third, moving average smoothing was applied to selected sensor features. Since the dataset was recorded every millisecond, the raw readings may contain high-frequency noise. A moving average window of 1000 rows was used, corresponding approximately to one second. This reduced short-term noise while preserving meaningful temporal trends.

Fourth, feature scaling was applied before training the deep learning models. MinMaxScaler was used to transform input features into a normalized range. This is important for neural networks because features with different scales can slow down training or cause unstable optimization.

Finally, time-series sequences were constructed for LSTM and TCN models. For TCN, a sampling rate of 1000 was used so that each timestep in the model represented approximately one second instead of one millisecond. This allowed the model to observe a more meaningful time span.

4. METHODOLOGY

4.1 Overall Pipeline

The overall pipeline consisted of several stages. First, the raw Parquet files were loaded into Python using Pandas. Next, missing value analysis and preprocessing were performed. Time-based features were extracted from local Mars time, and selected numerical features were smoothed using moving average filtering. The data was then scaled and converted into model-specific input formats.

For MLP and XGBoost, the dataset was treated primarily as structured feature rows. For sequence models such as LSTM and TCN, the dataset was transformed into temporal input sequences. After training, predictions were generated for the test set and submitted to Kaggle for scoring.

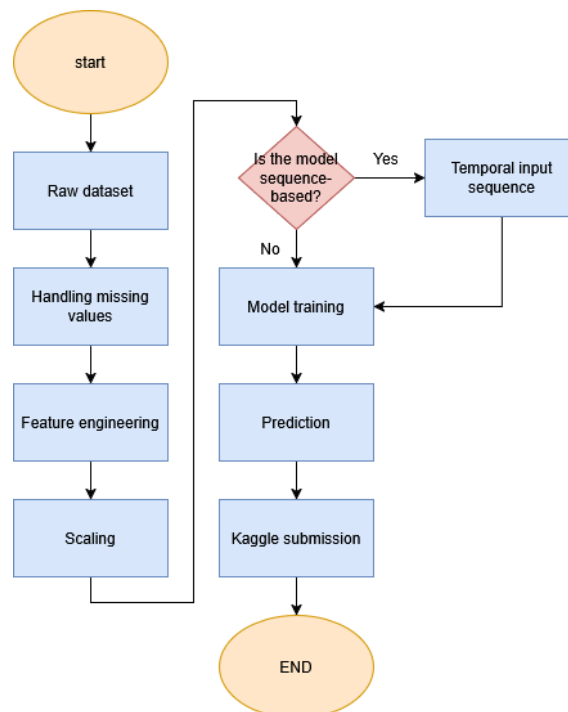


Figure 1 Overall experiment pipeline implemented throughout the project

4.2 Baseline Model

A Multi-Layer Perceptron (MLP) was used as the baseline model. It was selected because it provides a simple neural network approach for regression problems and can learn nonlinear relationships between input features and atmospheric pressure.

The MLP consisted of fully connected dense layers. Unlike LSTM and TCN, the MLP does not explicitly process temporal sequences. Therefore, it served as a useful baseline to evaluate whether more advanced time-series architectures were necessary.

The use of MLP as the baseline allowed the project to compare simple feedforward learning against more complex models that attempt to capture temporal dependencies.

4.3 Primary Model Architectures

1. XGBoost

XGBoost was used as a tree-based comparison model. It was selected because it is commonly effective for structured tabular data and can model nonlinear feature interactions. XGBoost builds an ensemble of decision trees, where each new tree attempts to correct the errors of the previous trees.

Although XGBoost can perform well on tabular datasets, it does not naturally model time-series order unless additional temporal features are manually engineered. Therefore, it was used to compare the effectiveness of tabular learning against sequence-based deep learning approaches.

2. LSTM

LSTM was implemented as a deep learning sequence model. The LSTM architecture consisted of stacked LSTM layers followed by dropout layers and a dense output layer. The purpose of using LSTM was to allow the model to learn temporal dependencies from previous observations.

LSTM is suitable for sequential data because it uses memory cells and gating mechanisms to retain information across timesteps. However, because the dataset was recorded at millisecond frequency, the effective time span represented by short sequences was limited unless sampling or longer sequence windows were applied.

3. TCN

TCN was implemented as a convolution-based time-series model. The TCN architecture used stacked residual blocks with causal and dilated one-dimensional convolutional layers. Causal convolution ensures that the model only uses past information when predicting the current pressure value, preventing information leakage from future observations.

Dilated convolution allows the model to view observations from further back in time without requiring extremely long input sequences. The dilation rates used were 1, 2, 4, 8, 16, and 32. This allowed the model to learn both short-term and longer-term temporal patterns.

Residual connections were included to improve training stability. They allow information to pass through the network more easily and reduce the risk of degradation when stacking multiple convolutional layers.

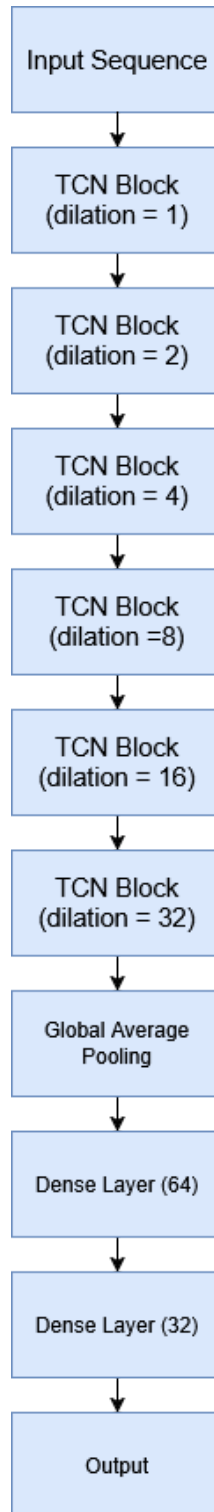


Figure 2 Pipeline architecture for TCN

4.4 Training Configuration

The models were trained using configurations suitable for their respective architectures. The MLP and XGBoost models used structured input features, while LSTM and TCN used sequence-based inputs.

The main TCN training configuration was as follows:

- Timesteps: 60
- Sampling rate: 1000
- Moving average window: 1000
- TCN filters: 64
- Kernel size: 3
- Dropout rate: 0.10
- Learning rate: 0.001
- Batch size: 1024
- Optimizer: Adam
- Loss function: Mean Squared Error

The sampling rate of 1000 was selected because the dataset was recorded every millisecond. Without sampling, 60 timesteps would only represent 60 milliseconds, which is too short for meaningful atmospheric pressure patterns. With a sampling rate of 1000, 60 timesteps represent approximately 60 seconds of historical context.

4.5 Evaluation Metrics

The primary evaluation was based on the Kaggle leaderboard score. During neural network training, validation loss was monitored using mean squared error. Mean absolute error was also observed as an additional metric to understand prediction error magnitude.

The Kaggle leaderboard score was treated as the final performance measure because it evaluated predictions on the hidden test set. However, validation performance was also useful for monitoring training behaviour and identifying overfitting.

5. EXPERIMENTAL SETUP

5.1 Computing Environment

The experiments were conducted using Jupyter notebook. The main libraries used include:

- Pandas for data loading and preprocessing
- NumPy for numerical operations
- Scikit-learn for scaling
- TensorFlow/Keras for MLP, LSTM, and TCN models
- XGBoost for tree-based comparison modelling
- Matplotlib for visualisation
- Garbage collection utilities for memory management

The dataset size required careful memory handling. Intermediate dataframes were deleted after conversion into NumPy arrays to reduce memory usage. Batch processing and TensorFlow dataset generators were used to train deep learning models more efficiently.

5.2 Experiment Log

Several experiments were conducted throughout the project. The MLP model was first implemented as the baseline neural network model. XGBoost was then tested as a tree-based machine learning approach. LSTM was implemented to investigate whether sequential learning could improve prediction performance.

The Temporal Convolutional Network (TCN) model was included because it offers several advantages for time-series forecasting, including the ability to capture both short-term and long-term dependencies through dilated causal convolutions, efficient parallel computation, and stable training through residual connections. Previous research on Martian weather forecasting has also highlighted the potential of TCN for surface pressure prediction, which motivated its inclusion as one of the architectures investigated in this project (Upreti et al., 2025).

Additional experiments were conducted by modifying the moving average window, time features, sequence construction, and learning rate.

5.3 Hyperparameter Tuning

Hyperparameter tuning was performed experimentally by changing one or a small number of parameters at a time. The main parameters tested include:

- Moving average window size
- Learning rate
- Number of timesteps
- Sampling rate
- Dropout rate

For LSTM, a moving average window of 1000 was explored because the dataset was recorded at millisecond frequency. For TCN, different sampling rates were investigated,

including a sampling rate of 1000, which allows each timestep to represent approximately one second rather than consecutive millisecond observations.

Different learning rates were also tested for TCN, including 0.001 and 0.0003, to examine how the optimization process and convergence behaviour changed under different training configurations.

6. RESULTS AND ANALYSIS

6.1 Model Performance Comparison

Below is the summarization of the top scores achieved by each model.

Table 1 Top scores achieved for each model

Model	Kaggle Score
MLP	8803.98
XGBoost	8791.79
LSTM	7458.85
TCN	2352.01

The results show that the TCN model produced the strongest performance among the models tested. The MLP baseline provided a reference point for evaluating more complex models. XGBoost performed reasonably well as a tabular model, while LSTM improved performance by incorporating temporal information. However, TCN achieved the best result because it efficiently captured temporal patterns using dilated causal convolutions.

6.2 Kaggle Leaderboard Standings

The best Kaggle submission was produced by the TCN model, achieving an approximate score of 2352.01 with a standing at 4th place as of Thursday 11th June . This was the strongest result obtained by the team during the competition. The leaderboard result demonstrated that the TCN architecture was better suited to the hidden test data compared to the other models tested.

#	Team	Members	Score	Entries	Last	Join
1	lurs lurs		0.000000000000	4	8h	
2	Alan Scott Encinas		29.146599904319	3	3d	
3	Matthew Degtyar		86.143995780281	6	1mo	
4	Penunggang Kaggle		2352.011714697970	22	1h	

Figure 3 Final position obtained in the Kaggle competition

6.3 Prediction Visualisations

Prediction visualisation was generated for the TCN model by comparing the predicted pressure values with the actual pressure values on the validation set.

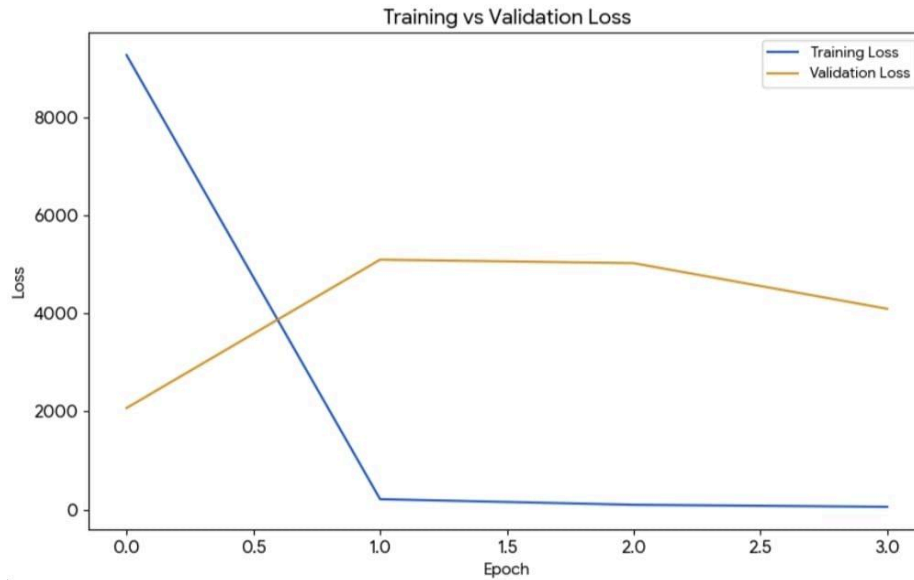


Figure 4 Plot for training VS Validation for TCN

This figure helps understanding whether the model captures the overall temporal behaviour of atmospheric pressure or only minimizes numerical error.

6.4 Error Analysis

The main source of error may be related to noise, missing values, and distribution differences between the training, validation, and hidden test data. Since the dataset was recorded at millisecond intervals, adjacent observations were highly correlated. This means that the raw number of rows may overestimate the true amount of independent information available.

Another potential source of error is feature selection. During preprocessing, columns with high missing percentages were removed. However, some features with many missing values may still contain important predictive information when available. A more rigorous feature selection method would involve calculating feature importance before dropping columns.

Overfitting was also observed in the TCN experiment. The best result occurred at the first epoch, while later epochs did not improve performance. This suggests that the model quickly learned the dominant pressure dynamics, and additional training may have started fitting noise or dataset-specific patterns.

6.5 Discussion

The experimental results suggest that model architecture selection was critical for this problem. The MLP baseline was useful for establishing a simple neural network reference, but it did not explicitly model temporal dependencies. XGBoost was a strong tabular model, but it was unable to fully capture the temporal nature of the dataset without additional sequential feature engineering.

LSTM improved performance by using sequential information, but it still underperformed compared with TCN due to several limitations. Although it is designed for sequence modelling, it processes data sequentially, meaning that each timestep depends on the computation of the previous timestep. This sequential nature limits parallelization during training and increases computational cost, especially when long input sequences are used. In this project, the original data were sampled at millisecond intervals, resulting in highly redundant sequences with many timesteps containing similar information. Even after downsampling, the model still needed to learn dependencies across relatively long sequences. LSTMs can struggle in such situations because important information must be propagated through many recurrent steps, increasing the risk of information degradation and making optimisation more difficult.

TCN performed best because it combines causal convolution, dilated convolution, and residual connections. Causal convolution preserves the correct temporal order by preventing future data leakage. While dilated convolution expands the receptive field, allowing the model to learn longer-term patterns efficiently. On the other hand, residual connections improve training stability and enable deeper architectures without suffering from vanishing gradient problems. Unlike LSTM, TCN can process all timesteps in parallel and capture long-range dependencies through convolutional filters, making it more computationally efficient and better suited to the characteristics of this dataset.

The millisecond sampling rate was also an important factor. Without sampling, a sequence of 60 timesteps would only represent 60 milliseconds. By using a sampling rate of 1000, the model observed approximately 60 seconds of historical information. This allowed the TCN to capture more meaningful temporal behaviour.

Overall, the results show that understanding the dataset structure was as important as selecting the model. The best performance was achieved only after recognizing that the dataset was highly redundant due to its sampling frequency and that a convolutional sequence model was more suitable than purely feedforward, recurrent, or tabular approaches.

7. REFLECTION

7.1 Challenges Encountered

Several challenges were encountered during the project. The first challenge was the size of the dataset. With approximately 9 million rows, training models required careful memory management and efficient preprocessing.

The second challenge was missing data. Some columns contained a large percentage of missing values. Initially, columns were dropped based on missing percentage thresholds. However, this approach may remove useful features. A better approach would be to calculate feature importance before removing features.

The third challenge was model selection. MLP, XGBoost, LSTM, LightGBM, ensemble models, and TCN were tested. Some theoretically reasonable changes did not improve performance, showing that practical experimentation was necessary.

The fourth challenge was overfitting. The TCN model achieved its best score at the first epoch, suggesting that it learned useful patterns quickly but did not benefit from further training.

7.2 Lessons Learned

One important lesson is that a large dataset does not always mean a large amount of independent information. Since the data was recorded at millisecond intervals, many adjacent rows were highly similar. Therefore, the sampling strategy was crucial.

Another lesson is that feature selection should be data-driven. Dropping columns only based on missing percentages may remove important information. Feature importance analysis should be performed before deciding which columns to remove.

The project also showed that model architecture must match the nature of the data. MLP and XGBoost were useful for baseline and comparison, but sequence-based models were more suitable for learning temporal patterns. TCN was more effective because it captured temporal patterns through dilated convolution. This was consistent with findings from previous Martian weather forecasting research.

Finally, the project highlighted the importance of experimentation. The best result was achieved after testing several models and learning from their limitations.

8. CONCLUSION AND FUTURE WORK

8.1 Conclusion

This project investigated the prediction of Martian atmospheric pressure using a large multivariate time-series dataset. Several models were tested, including MLP, XGBoost, LSTM, and TCN.

The MLP model served as the baseline neural network, while XGBoost was used as tabular comparison models. LSTM was implemented to capture sequential dependencies, and TCN was explored as a convolution-based time-series model. The best result was achieved using TCN, which showed that convolution-based temporal modelling was effective for this dataset.

The success of TCN can be attributed to its ability to capture temporal patterns efficiently using causal and dilated convolutions. The use of sampling rate also allowed the model to observe a meaningful time window despite the millisecond-level sampling frequency of the dataset.

Overall, the project successfully demonstrated that deep learning architectures designed for temporal data can significantly improve Martian pressure prediction performance.

8.2 Future Work

Future work should focus on improving feature selection and model generalization. Before dropping columns with high missing percentages, feature importance analysis should be performed. This would allow important but partially missing features to be retained.

Further experiments could also explore more advanced time-series models such as TiDE, N-BEATS, Transformer-based models, or hybrid TCN-LSTM architectures. Different sampling rates and moving average windows could be tested more systematically.

Another improvement would be to perform more robust validation using multiple time-based splits. This would help determine whether the model generalizes well across different time periods and sensor conditions.

Finally, additional visual analysis should be conducted to better understand the prediction errors. Plotting actual versus predicted pressure over different time intervals may reveal whether errors occur during specific atmospheric conditions or sensor states.

REFERENCES

1. Uprety, S., Bennaceur, A., Gavidia-Calderon, C., Holmes, J. A., Patel, M. R., & Rajendran, K. (2025). *Weather Prediction on Mars as a Multivariate Time Series Forecasting Problem*. <https://www.wunderground.com/>